Swift

Trilingual multimodal AI support-ticket system for banking — project overview · 14 July 2026

> It's an AI support-ticket system for a bank. Customers write in (Sinhala, English, or Tamil — typed, spoken as a voice note, or as a photo of a bank slip/receipt/error screen). The system figures out what language it is, turns audio/images into text, decides what the ticket is about and how urgent it is, and then either answers it automatically (for routine questions like "what's the FD rate?") or hands it to a human agent (for anything serious like fraud). Agents work from a dashboard that shows the queue, the AI's guesses, and an AI-drafted reply they can approve, edit, or reject.

The Two Users

- **Customer** — writes/speaks/photographs a question in their own language, gets a fast answer or gets told a human will follow up.
- **Agent** — works a queue, reviews AI-labeled tickets, approves/edits AI-drafted replies, corrects wrong labels.

Core Features (what gets built)

1. **Multi-way ticket intake** — web chat widget, Telegram, WhatsApp. Accepts text, voice recordings, and images in the same ticket.
2. **Understanding the ticket** — audio gets transcribed (speech-to-text), images get read (OCR pulls out amounts/dates/error text from bank slips), then everything becomes one block of text.
3. **Language detection** — is this Sinhala, English, or Tamil — including "Singlish/Tanglish" (Sinhala/Tamil typed in English letters).
4. **Classification** — labels predicted for every ticket: category (cards, fraud, loans, etc.), priority (low→urgent), sentiment (positive/neutral/negative), each with a confidence score.
5. **Auto-answering (RAG)** — for routine questions (rates, fees, KYC), the system searches a knowledge base and drafts a grounded answer in the customer's own language. Can be read aloud (text-to-speech).
6. **Account lookups** — for "did my transfer go through" type questions, it checks a (mock, for now) banking database — never invents numbers.
7. **Escalation / safety net** — fraud, high-priority, or angry-customer tickets are never auto-answered. They always go to a human.

8. **Agent dashboard** — ticket queue, stats, charts (volume, categories, sentiment, language, input type).
9. **Ticket list + detail pages** — search/filter/sort tickets; open one to see the conversation, attachments, labels, and the suggested reply with approve/edit/reject buttons.
10. **Knowledge base management** — agents add/edit rate sheets, fee schedules, FAQs; system re-indexes them for search.
11. **Agent corrections feed back into retraining** — if an agent fixes a wrong label, that correction is saved and can be used later to make the model better (done manually/offline, not automatic).

What "Testing" Means Here (the research side)

This isn't just app testing — a big chunk of the project is comparing different AI models against each other to see what works best for Sinhala/Tamil/English banking text:

- Try several classifier models (a simple baseline, mBERT, XLM-R, Sinhala-specific and Tamil-specific models, even asking an LLM directly) and measure which one is most accurate per language.
- Try different ways of handling "Singlish/Tanglish" typed input.
- Try different speech-to-text and OCR models and measure how much transcription/OCR errors hurt accuracy downstream.
- Try different reply-writing LLMs and check they answer correctly and in the right language, especially checking if Sinhala output quality is good enough.
- Report results broken down by task × language × script × input type — nothing gets averaged away/hidden.

The end goal of that side is an honest evaluation report, not just a working demo.

Explicitly Out of Scope

No real money movement (read-only), no real bank connection (uses fake mock account data), no live phone calls (only uploaded recordings), no multiple banks in one instance, no mobile app, no handwriting OCR, no automatic model retraining pipeline (retraining is manual).

Current Status

Right now the repo only has planning documents and two reference PDFs — no frontend, no backend, no models wired up yet. The build plan itself says phases are "pending" — they haven't been broken into concrete steps yet.

Generated from the project's context docs (project-overview.md, architecture.md, model-research.md, build-plan.md, progress-tracker.md) in the Swift repository.